Improving Fine-Grained Manipulation through Demonstration Expansion and Execution Design

Jingxu Gu **Dongze Lian**[†]
Nanjing University
jingxugu@smail.nju.edu.cn liandz@nju.edu.cn

Abstract: Fine-grained manipulation requires a robot policy to follow instructions, remain robust to visual variations, and translate predicted actions into precise robot control. We present Demonstration Expansion and Execution Design (DEED), a framework for adapting a pretrained vision-language-action policy to the five MetaFine tasks through structured demonstration expansion and execution design. DEED generates demonstrations for different instructions from the same initial scene and renders each physical trajectory under different camera positions and lighting conditions while preserving its original states and action targets. A hierarchical sampling strategy balances instructions, trajectories, and visual conditions during supervised adaptation. At deployment, DEED converts predicted action chunks into robot commands using inverse kinematics, gripper mappings selected for each task, and closed-loop replanning. DEED achieves an official Overall score of 85.55%. Fixed-policy evaluations show that performance varies with execution horizon, denoising budget, and gripper mapping.

Keywords: Fine-grained manipulation, vision-language-action models, imitation learning, visual augmentation

1 Introduction

Language-conditioned manipulation requires a robot to identify the object, part, or motion specified by an instruction and translate it into precise spatial control [1, 2]. Recent Vision-Language-Action (VLA) policies combine pretrained visual-language representations with robot action learning and have demonstrated broad transfer across tasks and robot embodiments [3, 4, 5, 6, 7]. However, fine-grained manipulation remains challenging because it jointly requires instruction-sensitive target selection, robustness to visual variations, and accurate conversion from predicted actions to robot control.

MetaFine [8, 9] evaluates these capabilities through five tasks covering part-level grasping, directional motion, switch toggling, object sorting, and letter insertion. Its evaluation considers three dimensions: *perception* under camera and illumination changes, *understanding* of instruction-conditioned behavior, and *controlled behavior* during task execution.

Directly fine-tuning a pretrained VLA policy on the provided demonstrations leaves three practical gaps. First, observing different instructions only in different scenes can lead to scene-instruction confounding. Second, limited camera and illumination conditions provide insufficient visual coverage. Third, predicted action chunks may fail when the execution horizon, inverse kinematics conversion, or gripper semantics do not match the simulator interface. Addressing these gaps requires adapting both the demonstration distribution and the execution pipeline.

To address these gaps, DEED expands demonstrations across instructions and visual conditions, balances training exposure through hierarchical sampling, and adapts closed-loop execution to each task (Figure 1).

[†]Corresponding author.

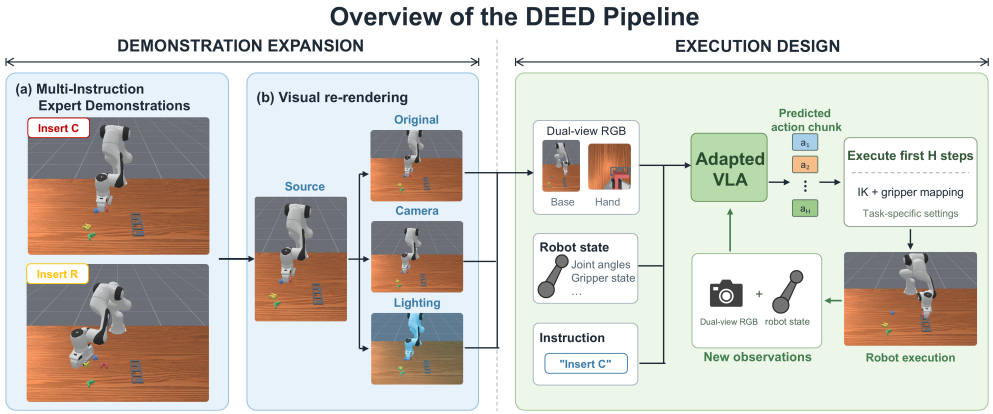


Figure 1: Overview of DEED (T5 illustrated). Paired demonstrations and visual re-rendering support adaptation; deployment executes an H-step prefix of each K-step prediction before re-observation.

Our contributions are: (1) A demonstration expansion strategy that varies instructions within the same scene and visual conditions for the same physical trajectory, supported by hierarchical sampling. (2) A task-aware execution pipeline that converts predicted action chunks into robot control through inverse kinematics, task-specific gripper mappings, and receding-horizon replanning. (3) A five-task evaluation achieving an official Overall score of 85.55%, together with fixed-policy studies of key execution choices.

2 Method

2.1 Instruction-paired demonstration expansion

Automated demonstration generation can expand imitation-learning datasets [10, 11]. Our inspection of the released T5 metadata found disjoint scene seeds across letter subsets [12]; pairing addresses the resulting opportunity for scene-based shortcuts. Let z index an initial scene with physical state x_0^z, and $\mathcal{L}_z$ denote its instruction family. The official expert planner $\mathcal{E}$ constructs

$$\tau_{z,\ell} = \mathcal{E}(x_0^z, \ell), \qquad \ell \in \mathcal{L}_z. \tag{1}$$

A saved physical trajectory is $\tau_{z,\ell} = (\ell, \{(x_t, s_t, a_t)\}_{t=0}^{T-1})$, where T is its length, x_t is the simulator state needed to reconstruct the scene (including robot and object configurations), s_t is the policy's proprioceptive input, and a_t is the expert action target. The future target chunk is $A_t = (a_t, \ldots, a_{t+K-1})$, with invalid entries masked at trajectory boundaries. The initial layout is shared, while the target and subsequent motion depend on ℓ. We retain a generated family only when all instruction variants succeed. Table 1 specifies each task's paired additions. Pairing provides different decisions under the same initial visual context.

2.2 Physics-consistent visual expansion

Re-rendering uses the benchmark's ManiSkill3/SAPIEN simulator [13, 14], following simulation-based visual randomization [15]. For a saved physical trajectory τ (omitting its scene and instruction subscripts), the renderer $\mathcal{R}$ restores each state x_t and produces base- and hand-camera images under visual parameters η, which specify camera and lighting settings:

$$I_t^\eta = \mathcal{R}(x_t; \eta), \qquad (s_t^\eta, A_t^\eta) = (s_t, A_t). \tag{2}$$

Only observation formation changes; proprioception and future action targets remain tied to the saved physical states. Simulator rendering preserves viewpoint-dependent geometry and action-label alignment. Camera and illumination profiles apply to all five tasks; T5 also uses combined camera and lighting perturbations (Table 1). Camera L1/L2/L3 use translation bounds of $0.03/0.06/0.12$ m and rotation bounds of $2/6/12°$; lighting half-widths are $0.10/0.25/0.40$ around ambient RGB 0.5.

T5 samples clean/camera/light/combined variants with probabilities $0.45/0.30/0.20/0.05$; combined rendering covers 120 base trajectories.

Expansion can otherwise overweight trajectories with many rendered copies. For T5, the data loader samples an augmentation category c (clean, camera, lighting, or combined), instruction ℓ, compatible base-trajectory index b, rendered variant v, and valid training-window start t hierarchically. The joint probability of selecting this training window is

$$p(c, \ell, b, v, t) = \frac{w_c}{|\mathcal{L}_c|\,|\mathcal{B}_{c,\ell}|\,|\mathcal{V}_{c,b}|\,|\mathcal{T}_{b,v}|}. \tag{3}$$

Here $w_c = p(c)$ is the prescribed category probability. The sets $\mathcal{L}_c$, $\mathcal{B}_{c,\ell}$, $\mathcal{V}_{c,b}$, and $\mathcal{T}_{b,v}$ contain, respectively, eligible instructions in category c, base trajectories for (c, ℓ), rendered variants of trajectory b in category c, and valid window starts in variant (b, v); $|\cdot|$ denotes set cardinality. Each choice after c is uniform within its conditional eligible set. Thus, p is the training-data sampling distribution, rather than the learned policy's action probability. Sampling base trajectories before rendered variants prevents extra visual copies from increasing a trajectory's sampling weight within a category and instruction. The final T5 pool contains 2,800 distinct successful trajectories and 7,840 records after rendering, including 2,000 new motions from 500 four-instruction scenes.

Table 1: Task coverage and execution settings.

Task	Paired instructions	Records	H	D	θ_g (mm)
T1	cap / body	6,400	10	5	35
T2	forward / left / right	3,300	25	2	15
T3	red / blue switch	2,080	25	5	—
T4	red / blue / green cube	2,940	25	5	35
T5	C / o / R / L	7,840	25	30	35

All tasks use pairing and camera/light L1–L3; only T5 additionally uses combined camera and lighting perturbations. Records include rendered variants. H: execution horizon; D: denoising steps; θ_g: closure threshold (—: continuous).

2.3 Supervised VLA adaptation

We adapt Xiaomi-Robotics-1 [16] with the task instruction, base and hand-eye RGB images, and joint/gripper state. The native state packs seven arm joint positions and the mean of two finger positions. The policy predicts $K = 30$ end-effector actions, with unused action slots masked. We retain Xiaomi-Robotics-1's architecture and released training implementation [16], including masks, weighting, and auxiliary terms; action generation uses flow matching [17]. Vision, language, action, and projection modules are updated with initialization-matched normalization and observation–action alignment. The final T5 continuation uses 12,000 BF16 updates (batch 4), a cosine learning-rate schedule from 3×10^{-6} to 5×10^{-7}, and 300 warmup updates.

2.4 Task-aware closed-loop execution

Local end-effector offsets are transformed into world-frame targets and converted into seven joint increments by inverse kinematics. Finger targets are clipped to $[-0.01, 0.04]$ m; values below the task threshold θ_g map to full closure (-0.01 m). T3 retains continuous gripper targets. Following action-chunking policies [18], the robot executes the first $H \leq K$ actions before obtaining fresh observations, as in receding-horizon control [19]. We select H, denoising count D, and closure threshold θ_g through task-specific evaluation rather than imposing one configuration on all contacts (Table 1).

3 Experiments

3.1 Setup and official results

For each task, we initialize the policy from a pretrained Xiaomi-Robotics-1 checkpoint and fine-tune it on task-specific demonstrations. Evaluation follows MetaFine's Perception (P), Understanding

(U), and Controlled Behavior (B) protocol [8, 9]. Here P denotes perception AUSC (area under the success curve). Task scores average P, U, B, and Overall averages the five task scores. Table 2 reports the organizer's final results [20]. DEED achieves 85.55% Overall, versus 43.01% for the published π_0 baseline (+42.54 percentage points; Figure 2) [20].

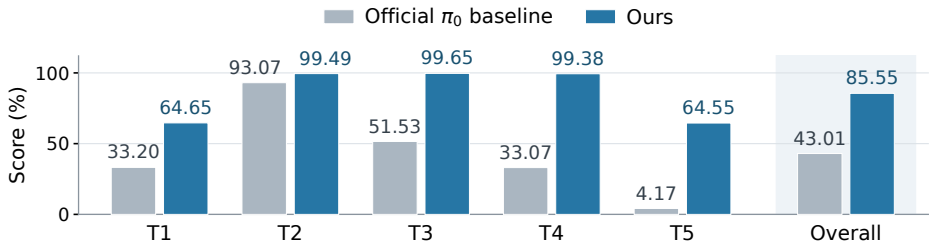


Figure 2: Comparison with the official π_0 baseline on the final leaderboard [20].

Table 2: Official final scores (%).

Task	P	U	B	Mean
T1	68.96	62.50	62.50	64.65
T2	98.47	100.00	100.00	99.49
T3	98.96	100.00	100.00	99.65
T4	98.15	100.00	100.00	99.38
T5	61.15	66.25	66.25	64.55
Average	85.14	85.75	85.75	**85.55**

Table 3: Local execution ablations (%; bold: deployed settings).

Task	Setting	P	U	SR
T1	$H = 25$	84.03	69.17	80.93
	$\boldsymbol{H = 10}$	**93.75**	**78.33**	**90.19**
	$H = 5$	85.56	65.83	81.30
T3	$D = 3$	98.13	97.50	98.06
	$\boldsymbol{D = 5}$	**98.13**	**97.50**	**98.06**
	$D = 10$	97.29	95.00	96.67

3.2 Fixed-policy component ablations

Table 3 uses the latest completed execution sweeps for the retained policies, with weights and normalization fixed within each comparison and flow-noise seed 0. The T1 sweep comprises three batches of 20 scenes, totaling 60 scenes and 540 episodes per horizon; the T3 sweep uses 40 new scenes and 360 episodes per denoising setting. SR pools two clean instruction episodes and seven perception episodes per scene with equal episode weights. Both sweeps preserve the public evaluator's environment reuse: T1 runs perception followed by understanding within each batch, whereas T3 evaluates the two modes in separate processes. Scene seeds are shared across settings, but subsequent physical initial states can differ because of environment history; these are aggregate comparisons rather than fully initial-state-matched counterfactuals. Changing H also changes the chunk-indexed flow-noise schedule.

In the T1 sweep, $H = 10$ scores higher than both $H = 25$ and $H = 5$; decreasing the execution window further is therefore not consistently beneficial. On T3, $D = 3$ and the deployed $D = 5$ tie, while $D = 10$ is lower by 0.83 points in P and 2.50 points in U. The expanded sweep did not include $D = 1$. Because the local sweep showed no degradation, $D = 5$ was submitted for organizer evaluation and retained for deployment. A 20-scene gripper study on an earlier partially adapted T1 policy yields $P = 27.92\%$ and $U = 14/40$ (35.00%) with continuous targets, versus $P = 57.92\%$ and $U = 22/40$ (55.00%) with full closure. Across eight expert-screened T5 scenes, correct instructions succeed in 21/32 trials (65.63%) versus 0/32 (0%) under cyclic letter permutation, with all initial-state and image pairs matched.

4 Conclusion

We present DEED, a framework that combines demonstration expansion with task-aware closed-loop execution for fine-grained manipulation. DEED generates instruction and visual variations while preserving state-action consistency, and translates predicted action chunks into robot control through inverse kinematics, task-specific gripper mappings, and receding-horizon replanning. It achieves an official Overall score of 85.55% across the five MetaFine tasks, while fixed-policy evaluations highlight the importance of execution configuration.

References

[1] M. Shridhar, L. Manuelli, and D. Fox. CLIPort: What and where pathways for robotic manipulation. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 894–906, 2022. URL https://proceedings.mlr.press/v164/shridhar22a.html.

[2] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. *arXiv preprint arXiv:2112.03227*, 2021. URL https://arxiv.org/abs/2112.03227.

[3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv preprint arXiv:2307.15818*, 2023. URL https://arxiv.org/abs/2307.15818.

[4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024. URL https://arxiv.org/abs/2406.09246.

[5] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024. URL https://arxiv.org/abs/2410.24164.

[6] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An Open-Source Generalist Robot Policy. *arXiv preprint arXiv:2405.12213*, 2024. URL https://arxiv.org/abs/2405.12213.

[7] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. *arXiv preprint arXiv:2310.08864*, 2023. URL https://arxiv.org/abs/2310.08864.

[8] H.-Y. Xu, P. Zhang, Z. Ge, X. Hao, S. Belongie, X. Geng, Y. Peng, and X.-S. Wei. Beyond binary success: A diagnostic meta-evaluation framework for fine-grained manipulation. *arXiv preprint arXiv:2605.19986*, 2026. URL https://arxiv.org/abs/2605.19986.

[9] Hiangx Robotics. MetaFine-Challenge_CoRL: Participant guide and evaluation protocol. https://github.com/Hiangx-robotics/MetaFine-Challenge_CoRL, 2026.

[10] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations. *arXiv preprint arXiv:2310.17596*, 2023. URL https://arxiv.org/abs/2310.17596.

[11] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots. *arXiv preprint arXiv:2406.02523*, 2024. URL https://arxiv.org/abs/2406.02523.

[12] Hiangx Robotics. MetaFine_CoRL26 demonstration dataset. https://huggingface.co/datasets/hiangx/MetaFine_CoRL26, 2026.

[13] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T. kai Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su. ManiSkill3: GPU Parallelized Robotics Simulation and Rendering for Generalizable Embodied AI. *arXiv preprint arXiv:2410.00425*, 2024. URL https://arxiv.org/abs/2410.00425.

[14] F. Xiang, Y. Qin, K. Mo, Y. Xia, H. Zhu, F. Liu, M. Liu, H. Jiang, Y. Yuan, H. Wang, L. Yi, A. X. Chang, L. J. Guibas, and H. Su. SAPIEN: A SimulAted Part-based Interactive ENvironment. *arXiv preprint arXiv:2003.08515*, 2020. URL https://arxiv.org/abs/2003.08515.

[15] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. *arXiv preprint arXiv:1703.06907*, 2017. URL https://arxiv.org/abs/1703.06907.

[16] Xiaomi Robotics Team, J. Guo, P. Jin, J. Li, P. Li, Y. Li, F. Liu, W. Peng, O. Qin, Y. Su, N. Sun, Q. Sun, R. Suo, H. Wang, Y. Wang, R. Wu, C. Xia, L. Zhang, J. Zhao, G. Chen, W. Chen, X. He, B. Li, Q. Li, Z. Li, H. Qu, W. Song, D. Xiang, Y. Xie, P. Xu, H. Ye, W. Ye, H. Zhao, and Q. Zhou. Xiaomi-Robotics-1: Scaling Vision-Language-Action Models with over 100K Hours of Real-World Trajectories. *arXiv preprint arXiv:2607.15330*, 2026. URL https://arxiv.org/abs/2607.15330. Code: https://github.com/XiaomiRobotics/Xiaomi-Robotics-1.

[17] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.02747.

[18] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023. URL https://arxiv.org/abs/2304.13705.

[19] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. doi:10.15607/RSS.2023.XIX.026. URL https://doi.org/10.15607/RSS.2023.XIX.026.

[20] RoboFineMani Organizers. RoboFineMani 2026 Official Leaderboard. https://robofinemani2026.github.io/leaderboard.html, 2026. Updated 24 October 2026; accessed 28 October 2026.